Absolutely. Below is a **SQL vs PySpark query comparison cheat sheet** focused on the operations most commonly used in **Data Engineering, PostgreSQL, Databricks and interviews**.

1. Basic SELECT

Requirement	SQL	PySpark
Select all columns	`SELECT * FROM employee;`	`df.select("*")`
Select columns	`SELECT name, salary FROM employee;`	`df.select("name", "salary")`
Select distinct	`SELECT DISTINCT dept FROM employee;`	`df.select("dept").distinct()`
Rename column	`SELECT name AS employee_name FROM employee;`	`df.withColumnRenamed("name", "employee_name")`
Select expression	`SELECT salary * 12 AS annual_salary FROM employee;`	`df.select((col("salary") * 12).alias("annual_salary"))`

2. WHERE / FILTER

Requirement	SQL	PySpark
Basic filter	`SELECT * FROM emp WHERE salary > 50000;`	`df.filter(col("salary") > 50000)`
Multiple conditions	`WHERE salary > 50000 AND dept = 'IT'`	`df.filter((col("salary") > 50000) & (col("dept") == "IT"))`
OR	`WHERE dept = 'IT' OR dept = 'HR'`	`df.filter((col("dept") == "IT") | (col("dept") == "HR"))`
NOT	`WHERE NOT active = true`	`df.filter(~col("active"))`
IN	`WHERE dept IN ('IT','HR')`	`df.filter(col("dept").isin("IT", "HR"))`
NOT IN	`WHERE dept NOT IN ('IT','HR')`	`df.filter(~col("dept").isin("IT", "HR"))`
BETWEEN	`WHERE salary BETWEEN 40000 AND 80000`	`df.filter(col("salary").between(40000, 80000))`
NULL	`WHERE salary IS NULL`	`df.filter(col("salary").isNull())`
NOT NULL	`WHERE salary IS NOT NULL`	`df.filter(col("salary").isNotNull())`

3. ORDER BY

Requirement	SQL	PySpark
Ascending	`ORDER BY salary ASC`	`df.orderBy(col("salary").asc())`
Descending	`ORDER BY salary DESC`	`df.orderBy(col("salary").desc())`
Multiple columns	`ORDER BY dept, salary DESC`	`df.orderBy(col("dept"), col("salary").desc())`
Limit	`LIMIT 10`	`df.limit(10)`

4. DISTINCT / DROP DUPLICATES

Requirement	SQL	PySpark
Distinct rows	`SELECT DISTINCT * FROM emp`	`df.distinct()`
Distinct column	`SELECT DISTINCT dept FROM emp`	`df.select("dept").distinct()`
Remove duplicates	`SELECT DISTINCT *`	`df.dropDuplicates()`
Based on column	—	`df.dropDuplicates(["employee_id"])`

5. NULL Handling

Requirement	SQL	PySpark
COALESCE	`COALESCE(salary, 0)`	`coalesce(col("salary"), lit(0))`
NULLIF	`NULLIF(a,b)`	`when(col("a") != col("b"), col("a"))`
Replace NULL	`COALESCE(name,'Unknown')`	`coalesce(col("name"), lit("Unknown"))`
Remove NULL	`WHERE salary IS NOT NULL`	`df.filter(col("salary").isNotNull())`
Drop NULL rows	—	`df.na.drop()`
Fill NULL	—	`df.na.fill(0)`

6. CASE WHEN

Requirement	SQL	PySpark
Simple CASE	`CASE WHEN salary > 50000 THEN 'High' ELSE 'Low' END`	`when(col("salary") > 50000, "High").otherwise("Low")`
Multiple conditions	`CASE WHEN salary >= 100000 THEN 'A' WHEN salary >= 50000 THEN 'B' ELSE 'C' END`	`when(col("salary") >= 100000, "A").when(col("salary") >= 50000, "B").otherwise("C")`
Conditional column	`SELECT CASE ... END AS grade`	`df.withColumn("grade", when(...))`

7. Aggregate Functions

Operation	SQL	PySpark
COUNT	`COUNT(*)`	`count("*")`
COUNT column	`COUNT(id)`	`count("id")`
COUNT DISTINCT	`COUNT(DISTINCT id)`	`countDistinct("id")`
SUM	`SUM(salary)`	`sum("salary")`
AVG	`AVG(salary)`	`avg("salary")`
MIN	`MIN(salary)`	`min("salary")`
MAX	`MAX(salary)`	`max("salary")`
STDDEV	`STDDEV(salary)`	`stddev("salary")`

Example:

```sql
SELECT
    dept,
    COUNT(*) AS employee_count,
    SUM(salary) AS total_salary,
    AVG(salary) AS avg_salary
FROM employee
GROUP BY dept;
```

```python
df.groupBy("dept").agg(
    count("*").alias("employee_count"),
    sum("salary").alias("total_salary"),
    avg("salary").alias("avg_salary")
)
```

8. GROUP BY / HAVING

Requirement	SQL	PySpark
Group by	`GROUP BY dept`	`df.groupBy("dept")`
Multiple columns	`GROUP BY dept,city`	`df.groupBy("dept","city")`
HAVING	`HAVING COUNT(*) > 10`	`.groupBy("dept").count().filter (col("count") > 10)`

Example:

```sql
SELECT dept, COUNT(*) AS cnt
FROM employee
GROUP BY dept
HAVING COUNT(*) > 10;
```

```
df.groupBy("dept") \
  .agg(count("*").alias("cnt")) \
  .filter(col("cnt") > 10)
```

9. JOIN

Join	SQL	PySpark
INNER	`INNER JOIN`	`.join(df2, condition, "inner")`
LEFT	`LEFT JOIN`	`.join(df2, condition, "left")`
RIGHT	`RIGHT JOIN`	`.join(df2, condition, "right")`
FULL	`FULL OUTER JOIN`	`.join(df2, condition, "full")`
LEFT ANTI	`LEFT ANTI JOIN`	`.join(df2, condition, "left_anti")`
LEFT SEMI	`LEFT SEMI JOIN`	`.join(df2, condition, "left_semi")`
CROSS	`CROSS JOIN`	`.crossJoin(df2)`

INNER JOIN

```
SELECT e.name, d.dept_name
FROM employee e
INNER JOIN department d
    ON e.dept_id = d.dept_id;
```

```
df_emp.join(
    df_dept,
    df_emp.dept_id == df_dept.dept_id,
    "inner"
).select(
    df_emp.name,
    df_dept.dept_name
)
```

10. UNION

Requirement	SQL	PySpark
UNION	`UNION`	`df1.union(df2).distinct()`
UNION ALL	`UNION ALL`	`df1.union(df2)`
Union by column name	—	`df1.unionByName(df2)`
Allow missing columns	—	`df1.unionByName(df2, allowMissingColumns=True)`

11. String Functions

Operation	SQL	PySpark
Upper	`UPPER(name)`	`upper("name")`
Lower	`LOWER(name)`	`lower("name")`
Length	`LENGTH(name)`	`length("name")`
Trim	`TRIM(name)`	`trim("name")`
Left trim	`LTRIM(name)`	`ltrim("name")`
Right trim	`RTRIM(name)`	`rtrim("name")`
Substring	`SUBSTRING(name,1,5)`	`substring("name",1,5)`
Replace	`REPLACE(name,'A','B')`	`regexp_replace("name","A","B")`
Concatenate	`CONCAT(first,last)`	`concat("first","last")`
Concatenate separator	`CONCAT_WS(' ',first,last)`	`concat_ws(" ","first","last")`
LIKE	`name LIKE 'A%'`	`col("name").like("A%")`
Regex	`name ~ '^[A-Z]'` PostgreSQL	`col("name").rlike("^[A-Z]")`

12. Date Functions

Requirement	SQL	PySpark
Current date	`CURRENT_DATE`	`current_date()`
Current timestamp	`CURRENT_TIMESTAMP`	`current_timestamp()`
Extract year	`EXTRACT(YEAR FROM dt)`	`year("dt")`
Extract month	`EXTRACT(MONTH FROM dt)`	`month("dt")`
Extract day	`EXTRACT(DAY FROM dt)`	`day("dt")`
Add days	`dt + INTERVAL '7 days'`	`date_add("dt", 7)`
Subtract days	`dt - INTERVAL '7 days'`	`date_sub("dt", 7)`
Difference	`date2 - date1`	`datediff("date2","date1")`
Month difference	`AGE()` / date calculations	`months_between()`
Start of month	`DATE_TRUNC('month',dt)`	`trunc("dt","month")`
Format date	`TO_CHAR(dt,'YYYY-MM-DD')`	`date_format("dt","yyyy-MM-dd")`
Convert string to date	`TO_DATE(dt)`	`to_date("dt")`

13. Window Functions

This is **very important for SQL + PySpark interviews**.

Requirement	SQL	PySpark
ROW_NUMBER	`ROW_NUMBER() OVER(...)`	`row_number().over(window)`
RANK	`RANK() OVER(...)`	`rank().over(window)`
DENSE_RANK	`DENSE_RANK() OVER(...)`	`dense_rank().over(window)`
LAG	`LAG(salary) OVER(...)`	`lag("salary").over(window)`
LEAD	`LEAD(salary) OVER(...)`	`lead("salary").over(window)`
Running SUM	`SUM(salary) OVER(...)`	`sum("salary").over(window)`
COUNT window	`COUNT(*) OVER(...)`	`count("*").over(window)`

SQL

```sql
SELECT
    employee_id,
    dept,
    salary,
    ROW_NUMBER() OVER (
        PARTITION BY dept
        ORDER BY salary DESC
    ) AS rn
FROM employee;
```


PySpark

```python
window = Window.partitionBy("dept").orderBy(col("salary").desc())

df.withColumn(
    "rn",
    row_number().over(window)
)
```

14. Top N Records per Group

SQL	PySpark
`ROW_NUMBER() OVER(PARTITION BY dept ORDER BY salary DESC)`	`row_number().over(Window.partitionBy("dept").orderBy(col("salary").desc()))`

```sql
SELECT *
FROM (
    SELECT *,
           ROW_NUMBER() OVER(
               PARTITION BY dept
               ORDER BY salary DESC
           ) rn
    FROM employee
) x
WHERE rn <= 3;
```

```python
window = Window.partitionBy("dept") \
               .orderBy(col("salary").desc())

df.withColumn(
    "rn",
    row_number().over(window)
).filter(col("rn") <= 3)
```

15. LAG / LEAD

SQL

```
SELECT
    employee_id,
    salary,
    LAG(salary) OVER(
        PARTITION BY dept
        ORDER BY salary
    ) AS previous_salary
FROM employee;
```

PySpark

```
window = Window.partitionBy("dept").orderBy("salary")

df.withColumn(
    "previous_salary",
    lag("salary").over(window)
)
```


16. CTE

SQL	PySpark
`WITH cte AS (...)`	DataFrame variable
Multiple CTEs	Multiple DataFrames
Recursive CTE	No direct equivalent

SQL:

```
WITH high_salary AS (
    SELECT *
    FROM employee
    WHERE salary > 100000
)
SELECT *
FROM high_salary;
```

PySpark:

```
high_salary = df.filter(col("salary") > 100000)

high_salary.show()
```

17. Subquery

SQL	PySpark
Subquery	DataFrame transformation
Correlated subquery	Usually JOIN/window transformation
EXISTS	`left_semi` join
NOT EXISTS	`left_anti` join

SQL:

```
SELECT *
FROM employee e
WHERE EXISTS (
    SELECT 1
    FROM department d
    WHERE e.dept_id = d.dept_id
);
```

PySpark:

```
df_emp.join(
    df_dept,
    "dept_id",
    "left_semi"
)
```

18. EXISTS / NOT EXISTS

SQL	PySpark
`EXISTS`	`left_semi`
`NOT EXISTS`	`left_anti`

```sql
SELECT e.*
FROM employee e
WHERE NOT EXISTS (
    SELECT 1
    FROM resigned r
    WHERE e.employee_id = r.employee_id
);
```

```python
df_emp.join(
    df_resigned,
    "employee_id",
    "left_anti"
)
```


19. Pivot

SQL

```sql
SELECT *
FROM employee
PIVOT (
    SUM(salary)
    FOR dept IN ('IT','HR','Finance')
);
```

PySpark

```python
df.groupBy("location") \
  .pivot("dept") \
  .sum("salary")
```

20. Unpivot

SQL	PySpark
`UNPIVOT`	`stack()` / `expr()`

Example:

```python
df.selectExpr(
    "id",
    "stack(3, 'Jan', Jan, 'Feb', Feb, 'Mar', Mar) as (month, amount)"
)
```

21. Explode Array

SQL/PostgreSQL	PySpark
`UNNEST(array_col)`	`explode("array_col")`

SQL:

```
SELECT id, UNNEST(tags)
FROM employee;
```

PySpark:

```
df.select(
    "id",
    explode("tags").alias("tag")
)
```


22. Array Functions

Operation	SQL	PySpark
Array length	`array_length(arr,1)`	`size("arr")`
Explode	`UNNEST(arr)`	`explode("arr")`
Contains	`value = ANY(arr)`	`array_contains("arr", value)`
Sort array	`array_sort(arr)`	`sort_array("arr")`
Remove duplicates	`array_distinct(arr)`	`array_distinct("arr")`

23. JSON Functions

Requirement	PostgreSQL SQL	PySpark
JSON field	`data->>'name'`	`get_json_object("data","$.name")`
JSON object	`data->'address'`	`from_json()`
Parse JSON	JSON operators	`from_json()`
Convert to JSON	`row_to_json()`	`to_json()`

24. CAST / Data Types

Requirement	SQL	PySpark
Integer	`CAST(salary AS INTEGER)`	`col("salary").cast("int")`
Decimal	`CAST(salary AS DECIMAL(10,2))`	`col("salary").cast("decimal(10,2)")`
String	`CAST(id AS VARCHAR)`	`col("id").cast("string")`
Date	`CAST(dt AS DATE)`	`col("dt").cast("date")`
Timestamp	`CAST(dt AS TIMESTAMP)`	`col("dt").cast("timestamp")`

25. Add / Modify Column

Requirement	SQL	PySpark
New column	`SELECT salary*12 AS annual_salary`	`withColumn("annual_salary", col("salary")*12)`
Modify column	`UPDATE employee SET salary=salary*1.1`	`withColumn("salary", col("salary")*1.1)`
Multiple columns	`SELECT ...`	Multiple `withColumn()`

26. Delete / Filter Records

SQL	PySpark	
`DELETE FROM emp WHERE salary < 30000`	`df.filter(col("salary") >= 30000)`	
Physical DELETE	Database operation	Usually write filtered DataFrame
Delta DELETE	`DELETE FROM table WHERE...`	`DeltaTable.delete()`

Example:

```
from delta.tables import DeltaTable

delta_table = DeltaTable.forPath(spark, path)

delta_table.delete(
    "salary < 30000"
)
```

27. UPDATE

SQL	PySpark / Delta
`UPDATE table SET salary = salary * 1.1`	`DeltaTable.update()`
Normal DataFrame	No direct UPDATE
Delta Lake	Supports UPDATE

28. MERGE / UPSERT

SQL

```
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN
    UPDATE SET
        t.name = s.name,
        t.salary = s.salary
WHEN NOT MATCHED THEN
    INSERT (id, name, salary)
    VALUES (s.id, s.name, s.salary);
```

PySpark Delta

```
from delta.tables import DeltaTable

target = DeltaTable.forPath(spark, target_path)

target.alias("t").merge(
    source_df.alias("s"),
    "t.id = s.id"
).whenMatchedUpdateAll() \
 .whenNotMatchedInsertAll() \
 .execute()
```

29. Sampling

SQL	PySpark
`TABLESAMPLE`	`sample()`
`LIMIT`	`limit()`

```
df.sample(
    withReplacement=False,
    fraction=0.1
)
```


30. Rename / Drop Columns

Requirement	SQL	PySpark
Rename	`AS new_name`	`withColumnRenamed()`
Drop	`SELECT ...`	`drop()`
Multiple drop	—	`drop("col1","col2")`

31. SQL NULL Logic vs PySpark

Logic	SQL	PySpark
NULL	`IS NULL`	`isNull()`
NOT NULL	`IS NOT NULL`	`isNotNull()`
NULL replacement	`COALESCE()`	`coalesce()`
Conditional	`CASE WHEN`	`when().otherwise()`

32. Date Truncation

SQL	PySpark
`DATE_TRUNC('month', createddate)`	`date_trunc("month", "createddate")`
`DATE_TRUNC('year', createddate)`	`date_trunc("year", "createddate")`
`DATE_TRUNC('day', createddate)`	`date_trunc("day", "createddate")`

For PostgreSQL:

```
DATE_TRUNC('month', createddate)::date
```

PySpark:

```
date_trunc("month", col("createddate"))
```

33. Monthly Record Count

SQL

```sql
SELECT
    DATE_TRUNC('month', createddate)::date AS month,
    COUNT(*) AS record_count
FROM employee
GROUP BY 1
ORDER BY 1;
```

PySpark

```
df.groupBy(
    date_trunc("month", col("createddate")).alias("month")
).agg(
    count("*").alias("record_count")
).orderBy("month")
```


34. Duplicate Records

SQL

```sql
SELECT
    employee_id,
    COUNT(*) AS cnt
FROM employee
GROUP BY employee_id
HAVING COUNT(*) > 1;
```

PySpark

```python
df.groupBy("employee_id") \
  .count() \
  .filter(col("count") > 1)
```

35. Find Latest Record

SQL

```sql
SELECT *
FROM (
    SELECT *,
           ROW_NUMBER() OVER(
               PARTITION BY employee_id
               ORDER BY modifieddate DESC
           ) rn
    FROM employee
) x
WHERE rn = 1;
```

PySpark

```
window = Window.partitionBy("employee_id") \
               .orderBy(col("modifieddate").desc())

df.withColumn(
    "rn",
    row_number().over(window)
).filter(col("rn") == 1)
```

36. Running Total

SQL

```
SELECT
    employee_id,
    salary,
    SUM(salary) OVER(
        ORDER BY employee_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM employee;
```


PySpark

```
window = Window.orderBy("employee_id") \
    .rowsBetween(
        Window.unboundedPreceding,
        Window.currentRow
    )

df.withColumn(
    "running_total",
    sum("salary").over(window)
)
```

37. Previous / Next Record

Requirement	SQL	PySpark
Previous	`LAG(col)`	`lag("col").over(window)`
Next	`LEAD(col)`	`lead("col").over(window)`

38. Ranking

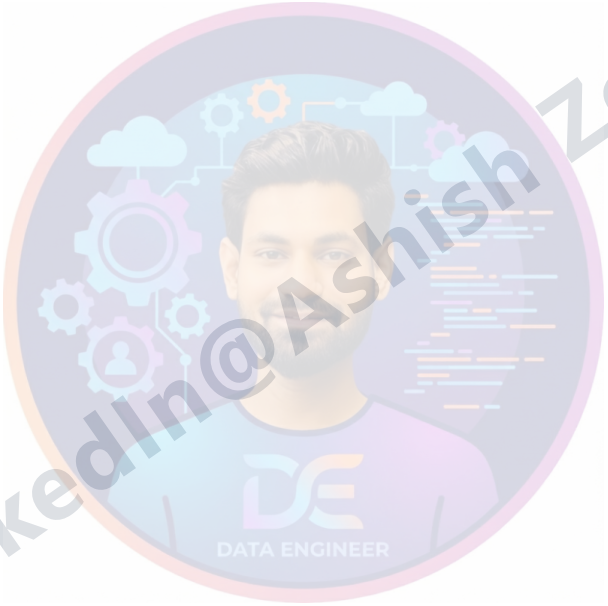
Ranking	SQL	PySpark
Row number	`ROW_NUMBER()`	`row_number()`
Rank	`RANK()`	`rank()`
Dense rank	`DENSE_RANK()`	`dense_rank()`

Example:

```
RANK() OVER (
    PARTITION BY dept
    ORDER BY salary DESC
)
```

```
rank().over(
    Window.partitionBy("dept")
          .orderBy(col("salary").desc())
)
```

39. SQL Functions vs PySpark Functions — Quick Reference



SQL Function	PySpark Function
COUNT()	count()
SUM()	sum()
AVG()	avg()
MIN()	min()
MAX()	max()
COALESCE()	coalesce()
UPPER()	upper()
LOWER()	lower()
TRIM()	trim()
LENGTH()	length()
SUBSTRING()	substring()
CONCAT()	concat()
CONCAT_WS()	concat_ws()
REPLACE()	regexp_replace()
CURRENT_DATE	current_date()
CURRENT_TIMESTAMP	current_timestamp()
EXTRACT()	year(), month(), day()
DATE_TRUNC()	date_trunc()
DATE_ADD()	date_add()
DATE_SUB()	date_sub()
DATEDIFF()	datediff()
ROW_NUMBER()	row_number()
RANK()	rank()
DENSE_RANK()	dense_rank()
LAG()	lag()
LEAD()	lead()

SQL Function	PySpark Function
`CASE WHEN`	`when().otherwise()`
`DISTINCT`	`distinct()`
`ORDER BY`	`orderBy()`
`GROUP BY`	`groupBy()`
`HAVING`	`filter()` after aggregation
`JOIN`	`join()`
`UNION ALL`	`union()`
`UNION`	`union().distinct()`
`LIMIT`	`limit()`
`UNNEST()`	`explode()`
`CAST()`	`cast()`
`IS NULL`	`isNull()`
`IS NOT NULL`	`isNotNull()`

40. SQL vs PySpark — Data Engineering Operations

Data Engineering Requirement	SQL	PySpark
Read table	`SELECT * FROM table`	`spark.table("table")`
Read CSV	`SELECT...` external table	`spark.read.csv()`
Read JSON	JSON functions/table	`spark.read.json()`
Read Parquet	External table	`spark.read.parquet()`
Write table	`INSERT INTO`	`df.write.saveAsTable()`
Write Parquet	`COPY` / external mechanism	`df.write.parquet()`
Append	`INSERT INTO`	`.mode("append")`
Overwrite	`TRUNCATE + INSERT`	`.mode("overwrite")`
Partition	`PARTITION BY`	`.partitionBy()`
Cache	DB-specific	`df.cache()`
Explain plan	`EXPLAIN`	`df.explain()`
Repartition	DB-specific	`repartition()`
Reduce partitions	—	`coalesce()`
Broadcast join	DB optimizer hint	`broadcast(df)`
Parallel processing	DB engine	Spark cluster

41. SQL Query → PySpark DataFrame Flow

A useful way to remember the conversion:

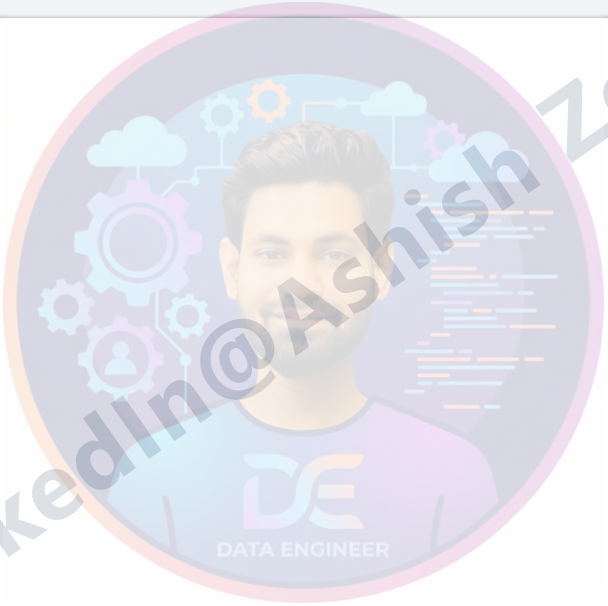
SQL	PySpark
SELECT	select()
FROM	DataFrame
WHERE	filter() / where()
GROUP BY	groupBy()
HAVING	filter() after aggregation
JOIN	join()
ORDER BY	orderBy()
DISTINCT	distinct()
LIMIT	limit()
UNION	union()
CASE	when()
WITH	DataFrame variable
Window	Window
INSERT	write
UPDATE	Delta update()
DELETE	Delta delete()
MERGE	Delta merge()

42. Complete Example

SQL

```sql
SELECT
    e.dept,
    e.employee_id,
    e.name,
    e.salary,
    CASE
        WHEN e.salary >= 100000 THEN 'HIGH'
        WHEN e.salary >= 50000 THEN 'MEDIUM'
        ELSE 'LOW'
    END AS salary_category,
    ROW_NUMBER() OVER (
        PARTITION BY e.dept
        ORDER BY e.salary DESC
    ) AS rn
FROM employee e
WHERE e.active = true
  AND e.salary IS NOT NULL;
```

PySpark



```
from pyspark.sql.functions import *
from pyspark.sql.window import Window

window = Window \
    .partitionBy("dept") \
    .orderBy(col("salary").desc())

result = (
    df
    .filter(
        (col("active") == True) &
        col("salary").isNotNull()
    )
    .withColumn(
        "salary_category",
        when(col("salary") >= 100000, "HIGH")
        .when(col("salary") >= 50000, "MEDIUM")
        .otherwise("LOW")
    )
    .withColumn(
        "rn",
        row_number().over(window)
    )
    .select(
        "dept",
        "employee_id",
        "name",
        "salary",
        "salary_category",
        "rn"
    )
)
```


The key mental mapping

SQL is declarative:

`SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY`

PySpark DataFrame is transformation-oriented:

`read → filter → withColumn → groupBy → agg → join → orderBy → write`

For a **Senior Data Engineer**, the most important SQL ↔ PySpark areas to master are **JOINs, GROUP BY, window functions, CTE/subqueries, CASE WHEN, date functions, NULL handling, deduplication, top-N, incremental loads, MERGE/UPSERT, partitioning, broadcast joins and execution plans**.